Knowledge graph — Software Design

Document ID	SDS-GRAPH-001
Revision	dfa6eda
Date	2026-11-10
Status	Controlled in git

Contents

1 Purpose 3
2 Design Outputs 3
3 Dependencies 5
4 Out of scope 6

Id: `SDS-GRAPH-001` · Context: `graph`

1 Purpose

This context owns the knowledge graph: the record — the spec, its test runs, git history, risks and checklists — projected into RDF for people and agents to query, browse and validate. It speaks of *projection* (one source of the record turned into facts, one named graph per source), the *vocabulary* (the classes and properties queries and agents rely on: the published language), and *gate rules* (shapes whose violations block and whose warnings inform). The graph is a read model: it is rebuilt from the spec, the documents and the results on demand, never edited, and never the source of a fact. It is open world: it states only what the record states, so a missing `rdm:exercises` link means *no run claimed the requirement*, never *unverified*; closed-world judgments are the gate rules' and the coded gates'. `graph` names it in commands and code.

2 Design Outputs

Component	Responsibility	Meets
Projection	Turns each source of the record into its own named graph, de-duplicated and in a stable order; writes sorted N-Quads; replaces the store on each build; answers SPARQL over the store or an in-memory projection; starts the read-only endpoint; adds checklists on request	item-06-001, item-06-002, item-06-003, item-06-008, item-06-009, item-06-011, item-06-012, item-06-013, item-06-014, item-06-015, item-06-016
Vocabulary	The classes and properties of the graph; projected as the ontology graph	item-06-001
Gate shapes	The gate rules as SHACL shapes: violations block exactly what the release gate blocks, warnings inform; plus the checklist data's own shapes	item-06-004, item-06-008, item-06-011, item-06-015, item-06-016
SHACL validation	`rdm graph validate`: runs the shipped and any user-supplied shapes over the union of the named graphs, reports each result	item-06-010

Component	Responsibility	Meets
	most severe first, and fails on any violation	
Explorer file	Writes the record as an AWS Graph Explorer file, every record node whatever its IRI, optionally leaving out chosen classes and what hangs only from them	item-06-005
Agent server	`rdm graph mcp`: the schema, query, trace and validate tools for agents, read-only, each answering from a projection rebuilt on that call	item-06-006, item-06-007, item-06-008, item-06-013

All of them run in the `rdm` command and need the optional extra `graph`; without it every `rdm graph` command exits 2 and names the extra.

How the components relate, in the direction of the arrows:

- The **Projection** declares its terms in the Vocabulary, and writes the explorer file with the Explorer file, which takes only the executions graph's name from the Projection, never a second read of the record. It builds the graph store, which the SPARQL endpoint (`rdm graph serve`) serves read-only to AWS Graph Explorer, its default graph the union of the named graphs, reading the store afresh for each query and refusing what the Agent server refuses.
- **SHACL validation** validates the Projection's graph and checks it with the Gate shapes.
- The **Agent server** projects the record afresh on each call; validates with SHACL validation; returns the Vocabulary for `schema`. It never uses the graph store, so an agent never reads a stale graph. The agent harness calls it over MCP stdio; RDM's own repository registers it for its agents.

The behaviour a reviewer needs to judge the design:

- **Named graphs.** The record graph holds the spec — its bounded contexts, user needs (text and source; from an adapter, also actors, surfaces, rules, guarantees and storm events), requirements (scenario title, rule, source, the needs they trace to and address) and an adapter's rules — and the controlled documents, each context's declaring document and part, and the documents' references; the executions graph the Allure results, when given; the git graph each document's latest landed commit and the commit the record was built at; the risks graph the register, with a finding about the register as a whole on each risk, or on a stand-in node for the register when it has none; the checklists and references graphs, on request; and the ontology graph the Vocabulary. The traceability matrix template is not projected: it is output, not record.
- **Runs claim requirements.** A run's id-shaped Allure `tag` labels become `rdm:exercises` links; a claim that names no requirement of the spec is a warning. The executions graph is read with the Allure reader the gates use.
- **Whole-record findings.** What the release gate blocks about the record as a whole (no spec, no requirement, an untagged scenario, an id on two scenarios, a spec file or result file that cannot be read) goes on the record's node as the gate words it, and a shape blocks each one,

so validation never passes what the gate blocks. A user need is unaddressed when no requirement `rdm:addresses` it, as the gate decides.
- **Identity.** Instance IRIs are `urn:dhf:<project>:<kind>/<id>`, the project being the repository's name unless another is given, so graphs from several repositories merge without collisions; clause, standard and checklist IRIs (`urn:rdm:clause:62304:5.6.2`) are project-independent.
- **Vocabulary reuse.** Standard terms are reused where they exist: `rdm:Requirement rdfs:subClassOf oslc_rm:Requirement` (OSLC Requirements Management), Dublin Core for identifiers, titles, descriptions and references, PROV-O for commits, authors and run times, SKOS for checklists. RDM terms (`rdm:UserNeed`, `rdm:BoundedContext`, `rdm:TestRun`, `rdm:Risk`, `rdm:tracesTo`, `rdm:addresses`, `rdm:exercises`, …) are minted only where none fits; the adapter's terms (`rdm:AlliumRule`, `rdm:coversRule`, `rdm:facing`, …) only when an adapter supplies the facts.
- **What is refused.** The Agent server's `query`, the SPARQL endpoint and `rdm graph query` refuse SPARQL Update, any `SERVICE` call outside literals, IRIs and comments (a comment ends at either line break), anything that does not parse as a query, and `FROM` or `FROM NAMED`, which would be ignored (a query names a graph with `GRAPH`); a variable or prefix named SERVICE is not a call. The Agent server's row limit is capped, and every tool is annotated read-only and idempotent. `trace` answers a requirement (its scenario, rule, needs, runs with their steps and attachments, source files and the risks it controls), a user need or a risk, matched against the ids the record declares and never spliced into a query; anything else is refused. SHACL validation exits 1 on a violation and 2 on a missing DHF or a shapes or checklist file that is missing or not RDF. An `--exclude` class the graph does not have is refused. The default branch is origin's HEAD, else `main` or `master`, else git's `init.defaultBranch`, else the only local branch.

3 Dependencies

A read model at the top layer (5) of the dependency rule: it depends on the contexts below it, and nothing but the composition root depends on it in code; the release context's gates action runs `rdm graph validate` as a CI step, not as an import. It projects:

- **specification**, through the spec reader: contexts, user needs, requirements and an adapter's rules;
- **test evidence**, through the Allure reader: results and labels, and the verified set;
- **risk**, through the Risk register: the register, the policy, the residual decision and the release gate's findings;
- **compliance**, through Gap analysis: the checklist reader, the built-in checklists and the key matcher;
- **release**, through the Release gate: its findings about the whole record, which the Projection puts on the record for the shapes;

and the shared kernel (ids, git history, frontmatter). It does not read publishing. No dependency breaks the rule.

4 Out of scope

Writing back: nothing the graph holds or derives changes the record. Pushing the graph to an external RDF store, versioning the vocabulary and gate shapes, and several projects in one graph are not yet in scope.